R4DS 11 – Joins

Keys

Primary and Foreign Keys

A **primary key** uniquely identifies each row. A **foreign key** links to another table's primary key.

Table	Primary key	Identifies
`airlines`	`carrier`	Airline
`airports`	`faa`	Airport
`planes`	`tailnum`	Plane
`weather`	`origin`, `time_hour`	Weather obs.

Foreign keys in `flights`: `carrier` → `airlines`, `tailnum` → `planes`, `origin`/`dest` → `airports$faa`, (`origin`, `time_hour`) → `weather`.

Checking Primary Keys

Verify uniqueness with `count()` + `filter()`:

```
planes |>
  count(tailnum) |> filter(n > 1)
```

```
#> # A tibble: 0 x 2
#> # i 2 variables: tailnum <chr>, n <int>
```

Also check for missing values:

```
planes |> filter(is.na(tailnum))
```

```
#> # A tibble: 0 x 9
#> # i 9 variables: tailnum <chr>, year <int>,
#> #   type <chr>, manufacturer <chr>, model <chr>,
#> #   engines <int>, seats <int>, speed <int>,
#> #   engine <chr>
#> ... [output truncated]
```

Surrogate keys: when no natural key exists, add one with `row_number()`:

```
flights |> mutate(id = row_number(), .before = 1)
```

Exercises (Keys)

1. What is the relationship between `weather` and `airports`?
2. If `weather` contained records for all US airports, what additional connection would it make to `flights`?
3. The `year`, `month`, `day`, `hour`, and `origin` variables almost form a compound key for `weather`, but one hour has duplicates. What's special about it?
4. How would you represent "special days" (e.g., Christmas) as a data frame? What would be its primary key?

Solutions (Keys)

1. `weather$origin` is a foreign key to `airports$faa`. Only the three NYC airports appear.
2. `weather` could also connect to `flights$dest`, providing weather at the destination.
3. The duplicate hour is likely the fall-back DST hour (November, when clocks go back — the same hour occurs twice).
4. A tibble with `month` and `day` as compound primary key. Connects to `flights` via those columns.

Mutating Joins

`left_join()` — The Most Common Join

A **mutating join** adds variables from one table to another by matching keys.

```
flights2 <- flights |>
  select(year, time_hour, origin, dest,
         tailnum, carrier)

flights2 |> left_join(airlines)
```

```
#> # A tibble: 336,776 x 7
#>     year time_hour           origin dest  tailnum
#>    <int> <dttm>              <chr>  <chr> <chr>
#>  1  2013 2013-01-01 05:00:00 EWR    IAH   N14228
#>  2  2013 2013-01-01 05:00:00 LGA    IAH   N24211
#>  3  2013 2013-01-01 05:00:00 JFK    MIA   N619AA
#>  4  2013 2013-01-01 05:00:00 JFK    BQN   N804JB
#>  5  2013 2013-01-01 06:00:00 LGA    ATL   N668DN
#> ... [output truncated]
```

`left_join()` keeps **all rows** in `x`, fills `NA` for non-matches.

`left_join()` — Adding Weather

```
flights2 |>
  left_join(weather |>
    select(origin, time_hour, temp, wind_speed))
```

```
#> # A tibble: 336,776 x 8
#>     year time_hour           origin dest  tailnum
#>    <int> <dttm>              <chr>  <chr> <chr>
#>  1  2013 2013-01-01 05:00:00 EWR    IAH   N14228
#>  2  2013 2013-01-01 05:00:00 LGA    IAH   N24211
#>  3  2013 2013-01-01 05:00:00 JFK    MIA   N619AA
#>  4  2013 2013-01-01 05:00:00 JFK    BQN   N804JB
#>  5  2013 2013-01-01 06:00:00 LGA    ATL   N668DN
#> ... [output truncated]
```

Non-matches produce NA for the new columns.

`left_join()` — Adding Plane Info

```
flights2 |>
  left_join(planes |>
    select(tailnum, type, engines, seats))
```

```
#> # A tibble: 336,776 x 9
#>     year time_hour           origin dest  tailnum
#>    <int> <dttm>              <chr>  <chr> <chr>
#>  1  2013 2013-01-01 05:00:00 EWR    IAH   N14228
#>  2  2013 2013-01-01 05:00:00 LGA    IAH   N24211
#>  3  2013 2013-01-01 05:00:00 JFK    MIA   N619AA
#>  4  2013 2013-01-01 05:00:00 JFK    BQN   N804JB
#>  5  2013 2013-01-01 06:00:00 LGA    ATL   N668DN
#> ... [output truncated]
```

Many planes have `NA` for `type`/`seats` — those tail numbers are missing from the `planes` table.

Specifying Join Keys with `join_by()`

Default: join on **all shared column names** (natural join). This can go wrong:

```
# year means different things in flights vs planes!
flights2 |> left_join(planes) |>
  select(tailnum, type) |> head(3)
```

```
#> # A tibble: 3 x 2
#>   tailnum type
#>   <chr>   <chr>
#> 1 N14228  <NA>
#> ... [output truncated]
```

Fix: specify the key explicitly:

```
flights2 |> left_join(planes, join_by(tailnum))
```

For **different column names**, use `==`:

```
flights2 |>
  left_join(airports, join_by(dest == faa))
```

Four Types of Mutating Joins

Simple example:

```
x <- tribble(~key, ~val_x, 1, "x1",
             2, "x2", 3, "x3")
y <- tribble(~key, ~val_y, 1, "y1",
             2, "y2", 4, "y3")
```

Join	Keeps rows from	Unmatched rows
`left_join(x, y)`	All of `x`	`y` gets `NA`
`right_join(x, y)`	All of `y`	`x` gets `NA`
`full_join(x, y)`	All of `x` and `y`	Both get `NA`
`inner_join(x, y)`	Only matches	Dropped

inner_join() and left_join()

```
x |> inner_join(y, join_by(key))
```

```
#> # A tibble: 2 x 3
#>     key val_x val_y
#>   <dbl> <chr> <chr>
#> 1     1 x1    y1
#> 2     2 x2    y2
```

```
x |> left_join(y, join_by(key))
```

```
#> # A tibble: 3 x 3
#>     key val_x val_y
#>   <dbl> <chr> <chr>
#> 1     1 x1    y1
#> 2     2 x2    y2
#> 3     3 x3    <NA>
```

full_join()

```
x |> full_join(y, join_by(key))
```

```
#> # A tibble: 4 x 3
#>     key val_x val_y
#>   <dbl> <chr> <chr>
#> 1     1 x1    y1
#> 2     2 x2    y2
#> 3     3 x3    <NA>
#> 4     4 <NA>  y3
```

Key 3 exists only in `x` → `val_y` is `NA`. Key 4 exists only in `y` → `val_x` is `NA`.

Filtering Joins: `semi_join()` and `anti_join()`

Filtering joins filter rows — they never add columns.

`semi_join(x, y)`: keep rows in `x` that **have** a match in `y`:

```
airports |>
  semi_join(flights2, join_by(faa == origin))
```

```
#> # A tibble: 3 x 8
#>   faa   name           lat   lon   alt    tz dst   tzone
#>   <chr> <chr>        <dbl> <dbl> <dbl> <dbl> <chr> <chr>
#> 1 EWR   Newark Lib~  40.7 -74.2    18    -5 A     Amer~
#> 2 JFK   John F Ken~  40.6 -73.8    13    -5 A     Amer~
#> 3 LGA   La Guardia   40.8 -73.9    22    -5 A     Amer~
#> ... [output truncated]
```

`anti_join(x, y)`: keep rows in `x` that have **no** match in `y`:

```
flights2 |>
  anti_join(airports, join_by(dest == faa)) |>
  distinct(dest)
```

```
#> # A tibble: 4 x 1
#>   dest
#>   <chr>
#> 1 BQN
```

Row Matching

Three outcomes for a row in `x`:

- **0 matches** — dropped (inner) or filled with `NA` (left/right/full)
- **1 match** — preserved as-is
- **>1 matches** — **duplicated** once per match

Many-to-many joins cause row explosion:

```
df1 <- tibble(key = c(1, 2, 2),
              val_x = c("x1", "x2", "x3"))
df2 <- tibble(key = c(1, 2, 2),
              val_y = c("y1", "y2", "y3"))
df1 |> inner_join(df2, join_by(key))
```

```
#> # A tibble: 5 x 3
#>     key val_x val_y
#>   <dbl> <chr> <chr>
#> 1     1 x1    y1
#> 2     2 x2    y2
#> 3     2 x2    y3
#> 4     2 x3    y2
#> ... [output truncated]
```

Exercises (Basic Joins)

1. Find the 48 hours with the worst delays over the year. Cross-reference with `weather`.
2. Find all flights to the top 10 most popular destinations.
3. Does every departing flight have corresponding weather data?
4. What do tail numbers missing from `planes` have in common? (Hint: one variable explains ~90%.)
5. Add origin **and** destination airport latitude/longitude to `flights`.
6. Compute average delay by destination, then plot on a US map.

Solutions (Basic Joins) — 1–3

1.

```
flights |>
  mutate(hour = dep_time %/% 100) |>
  group_by(year, month, day, hour) |>
  summarize(avg_delay = mean(dep_delay,
    na.rm = TRUE)) |>
  ungroup() |> slice_max(avg_delay, n = 48) |>
  left_join(weather)
```

2.

```
top_dest <- flights |>
  count(dest, sort = TRUE) |> head(10)
flights |> semi_join(top_dest, join_by(dest))
```

3. No — `flights |> anti_join(weather)` reveals some flights without matching weather.

Solutions (Basic Joins) — 4–6

4. Mostly `carrier == "MQ"` or `"AA"` — they report tail numbers differently.
5.

```
flights |>
  left_join(airports |> select(faa, lat, lon),
    join_by(origin == faa)) |>
  rename(origin_lat = lat, origin_lon = lon) |>
  left_join(airports |> select(faa, lat, lon),
    join_by(dest == faa)) |>
  rename(dest_lat = lat, dest_lon = lon)
```

Rename **after** the join to avoid ambiguity.

6.

```
flights |>
  group_by(dest) |>
  summarize(delay = mean(arr_delay, na.rm = TRUE)) |>
  inner_join(airports, join_by(dest == faa)) |>
  ggplot(aes(lon, lat, color = delay)) +
  borders("state") + geom_point() +
  coord_quickmap()
```

Non-Equi Joins

Cross Joins

`cross_join()` matches every row in `x` with every row in `y` (Cartesian product):

```
df <- tibble(name = c("John", "Simon",
                      "Tracy", "Max"))
df |> cross_join(df)
```

```
#> # A tibble: 16 x 2
#>    name.x name.y
#>    <chr>  <chr>
#>  1 John   John
#>  2 John   Simon
#>  3 John   Tracy
#>  4 John   Max
#>  5 Simon  John
#> ... [output truncated]
```

Useful for generating all permutations. Output has $n_x \times n_y$ rows.

Inequality Joins

Use <, <=, >, >= in `join_by()` instead of ==:

```
df <- tibble(id = 1:4,
  name = c("John", "Simon", "Tracy", "Max"))
df |> inner_join(df, join_by(id < id))
```

```
#> # A tibble: 6 x 4
#>    id.x name.x  id.y name.y
#>   <int> <chr>  <int> <chr>
#> 1     1 John       2 Simon
#> 2     1 John       3 Tracy
#> 3     1 John       4 Max
#> 4     2 Simon      3 Tracy
#> 5     2 Simon      4 Max
#> ... [output truncated]
```

Generates all **combinations** (not permutations) — avoids duplicates.

Rolling Joins

Match the **closest** row satisfying an inequality with `closest()`:

```
parties <- tibble(
  q = 1:4,
  party = ymd(c("2022-01-10", "2022-04-04",
                "2022-07-11", "2022-10-03")))

set.seed(123)
employees <- tibble(
  name = c("Alice", "Bob", "Carol", "Dan"),
  birthday = ymd(c("2022-03-15", "2022-06-20",
                   "2022-01-05", "2022-11-01")))

employees |>
  left_join(parties,
    join_by(closest(birthday >= party)))
```

```
#> # A tibble: 4 x 4
#>   name  birthday       q party
#>   <chr> <date>     <int> <date>
#> 1 Alice 2022-03-15     1 2022-01-10
#> 2 Bob   2022-06-20     2 2022-04-04
```

Overlap Joins

Three helpers for working with date ranges:

Helper	Condition
`between(x, y_lower, y_upper)`	`x >= y_lower & x <= y_upper`
`within(xl, xu, yl, yu)`	`xl >= yl & xu <= yu`
`overlaps(xl, xu, yl, yu)`	`xl <= yu & xu >= yl`

```
parties <- parties |>
  mutate(start = ymd(c("2022-01-01",
    "2022-04-04", "2022-07-11", "2022-10-03")),
    end = ymd(c("2022-04-03", "2022-07-10",
      "2022-10-02", "2022-12-31")))

employees |>
  inner_join(parties,
    join_by(between(birthday, start, end)))
```

Exercises (Non-Equi Joins)

1. Explain the difference in output between these two:

```
x |> full_join(y, join_by(key == key))
x |> full_join(y, join_by(key == key),
               keep = TRUE)
```

2. When checking party period overlaps, why is `q < q` needed in the `join_by()`? What happens without it?

Solutions (Non-Equi Joins)

1. Without `keep = TRUE`, the key columns are coalesced into one (showing the non-`NA` value). With `keep = TRUE`, both `key.x` and `key.y` are shown separately.
2. Without `q < q`, every interval overlaps with itself, giving spurious matches. The inequality ensures each pair is compared only once and excludes self-matches.